

KUBERNETES TRAINING · DAY 5

Node Maintenance & Advanced Scheduling

Hands-On Lab Guide

6 exercises · kubectl required · a running multi-node cluster

01

Node Maintenance

Cordon, drain, observe, uncordon

02

Node Selector

Label a worker, pin a pod to it

03

Node Affinity

Required vs. preferred placement rules

04

Pod Affinity & Anti-Affinity

Co-locate or spread replicas

05

Taints & Tolerations

Reserve nodes for specific workloads

06

Horizontal Pod Autoscaler

Scale replicas automatically under load

01

NODE MAINTENANCE

Cordon, Drain, Observe, Uncordon

Objective

Take a worker node out of service without dropping workloads, confirm the scheduler moved pods elsewhere, then return the node to the pool.

Step 1 — List your nodes

Identify a worker node to take offline.

```
kubectl get nodes
```

Step 2 — Cordon the node

Mark it unschedulable — running pods are not evicted yet.

```
kubectl cordon <node-name>
```

✓ Verify the cordon

`kubectl get nodes` — the node's STATUS column now shows **SchedulingDisabled**.

Step 3 — Drain the node

Evict all pods so they reschedule onto other nodes.

```
kubectl drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data
```

Step 4 — Observe pods move

Watch pods land on a different node in real time.

```
kubectl get pods -o wide --watch
```

✓ Confirm the move

Pods previously on the cordoned node now show a different value in the **NODE** column, and are **Running**.

Step 5 — Uncordon the node

Return the node to the schedulable pool.

```
kubectl uncordon <node-name>
```

✓ Verify the uncordon

kubectl get nodes — STATUS returns to **Ready** with no SchedulingDisabled flag.

★ Key takeaway

Cordon only blocks *new* scheduling. Drain is what actually evicts running pods — always cordon before you drain, and never drain without a plan for where the pods will land.

02

NODE SELECTOR

Pin a Pod to One Worker

Objective

Label a specific worker node, then force a pod to schedule only onto that node using **nodeSelector**.

Step 1 — Label a worker

Attach a key/value label the scheduler can match on.

```
kubectl label nodes <node-name> disk=ssd
```

✓ Verify the label

`kubectl get nodes --show-labels` — confirm `disk=ssd` appears for that node.

Step 2 — Write a pod spec with nodeSelector

Create `ssd-pod.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disk: ssd
  containers:
  - name: app
    image: nginx
```

Step 3 — Apply and check placement

Deploy the pod and confirm where it landed.

```
kubectl apply -f ssd-pod.yaml
kubectl get pod ssd-pod -o wide
```

✓ Confirm the pin

The **NODE** column for **ssd-pod** matches the node you labeled in Step 1.

★ **Key takeaway**

nodeSelector is an exact-match, all-or-nothing rule — if no node carries the label, the pod stays **Pending** forever. It's the simplest scheduling constraint, and the one every other mechanism in this guide builds on.

03

NODE AFFINITY

Required vs. Preferred Rules

Objective

Reproduce `nodeSelector`-style placement with **required** affinity, then add a softer **preferred** rule that the scheduler may override.

Step 1 — Label two nodes

Simulate two zones for the exercise.

```
kubectl label nodes <node-a> zone=zone-a
kubectl label nodes <node-b> zone=zone-b
```

Step 2 — Hard rule — required affinity

Create **required-affinity.yaml**. The pod will only schedule where the rule matches, exactly like `nodeSelector` but more expressive.

```
apiVersion: v1
kind: Pod
metadata:
  name: zone-a-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: zone
                operator: In
                values: ["zone-a"]
  containers:
    - name: app
      image: nginx
```

Step 3 — Apply and verify

```
kubectl apply -f required-affinity.yaml
kubectl get pod zone-a-pod -o wide
```

✓ Confirm placement

zone-a-pod lands only on the node labeled **zone=zone-a**.

Step 4 — Soft rule — preferred affinity

Create **preferred-affinity.yaml**. The scheduler tries zone-a first, ranked by weight, but still places the pod elsewhere if nothing matches.

```
apiVersion: v1
kind: Pod
metadata:
  name: prefer-zone-a-pod
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 80
          preference:
            matchExpressions:
              - key: zone
                operator: In
                values: ["zone-a"]
  containers:
    - name: app
      image: nginx
```

Step 5 — Apply and compare

Remove the zone-a label temporarily and re-apply to see the fallback behave differently from Step 3.

```
kubectl apply -f preferred-affinity.yaml
kubectl get pod prefer-zone-a-pod -o wide
```

★ Key takeaway

required = hard constraint, pod stays Pending if unmet. **preferred** = best-effort, pod still schedules if no node qualifies. Reach for preferred when you have a preference, not a hard requirement.

04

POD AFFINITY

Co-Locate or Spread Replicas

Objective

Use **podAffinity** to keep two related pods on the same node, then use **podAntiAffinity** to force replicas of the same app onto different nodes.

Step 1 — Deploy the anchor pod

This pod is the target that others will affine to.

```
kubectl run frontend-anchor --image=nginx --labels="app=frontend"
```

Step 2 — Add a pod with affinity to it

Create **frontend-affine.yaml** — schedules onto the same node as any pod labeled **app=frontend**.

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend-cache
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["frontend"]
          topologyKey: "kubernetes.io/hostname"
  containers:
    - name: cache
      image: redis
```

Step 3 — Apply and verify co-location

```
kubectl apply -f frontend-affine.yaml
kubectl get pods -o wide
```


✓ Confirm affinity

frontend-cache and **frontend-anchor** share the same value in the **NODE** column.

Step 4 — Scale a deployment with anti-affinity

Create **web-anti-affinity.yaml** — spreads replicas of the same app across separate nodes.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchLabels: {app: web}
              topologyKey: "kubernetes.io/hostname"
      containers:
        - name: web
          image: nginx
```

Step 5 — Apply and verify spread

```
kubectl apply -f web-anti-affinity.yaml
kubectl get pods -l app=web -o wide
```

✓ Confirm the spread

Each of the 3 **web** replicas shows a **different** value in the **NODE** column. If your cluster has fewer than 3 workers, one replica stays **Pending** — that's expected.

★ Key takeaway

Both rules match on *pod labels*, not node labels — that's what separates them from node affinity. Anti-affinity is the standard way to keep a single node failure from taking down every replica.

05

TAINTS & TOLERATIONS

Reserve a Node for a Workload

Objective

Taint a node so it repels ordinary pods, then demonstrate both the **NoSchedule** and **NoExecute** effects.

Step 1 — Taint a node — NoSchedule

Reserve the node for a specific workload only.

```
kubectl taint nodes <node-name> dedicated=gpu:NoSchedule
```

Step 2 — Try scheduling an ordinary pod

Deploy a plain pod with no toleration.

```
kubectl run plain-pod --image=nginx
```

✓ Confirm the repel

kubectl get pod plain-pod -o wide — it never lands on the tainted node; it schedules elsewhere or stays **Pending** if that's the only node.

Step 3 — Add a toleration

Create **gpu-toleration.yaml** so this pod is allowed on the tainted node.

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "gpu"
      effect: "NoSchedule"
  containers:
    - name: app
      image: nginx
```

Step 4 — Apply and verify

```
kubectl apply -f gpu-toleration.yaml
kubectl get pod gpu-pod -o wide
```

✓ Confirm tolerance

gpu-pod is the only pod able to land on the tainted node.

Step 5 — Demonstrate NoExecute

Change the taint effect — this evicts pods already running there that lack a matching toleration.

```
kubectl taint nodes <node-name> dedicated=gpu:NoSchedule-
kubectl taint nodes <node-name> dedicated=gpu:NoExecute
kubectl get pods -o wide --watch
```

✓ Confirm eviction

Any pod on that node without the toleration moves to **Terminating**, while **gpu-pod** (with a matching toleration) stays put.

★ Key takeaway

Taints live on the **node**; tolerations live on the **pod** — the inverse of affinity. **NoSchedule** blocks new pods only; **NoExecute** also evicts pods already running there.

06

HORIZONTAL POD AUTOSCALER

Scale Replicas Under Load

Objective

Install the Metrics Server, generate CPU load against a deployment, and watch the Horizontal Pod Autoscaler add replicas automatically.

Step 1 — Install Metrics Server

Deploy the cluster metrics pipeline HPA depends on.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

✓ Verify it's running

kubectl get deployment metrics-server -n kube-system — READY shows **1/1**. **kubectl top nodes** starts returning live CPU/memory figures once it's warmed up.

Step 2 — Deploy a workload with resource requests

HPA needs a CPU request to calculate percentage usage.

```
kubectl create deployment php-apache \
  --image=registry.k8s.io/hpa-example \
  --requests=cpu=200m
kubectl expose deployment php-apache --port=80
```

Step 3 — Create the autoscaler

Scale between 1 and 10 replicas, targeting 50% average CPU.

```
kubectl autoscale deployment php-apache \
  --cpu-percent=50 --min=1 --max=10
```

Step 4 — Generate CPU load

Run a busy loop that hammers the service continuously.

```
kubectl run load-generator --image=busybox --restart=Never -it -- \
  /bin/sh -c "while true; do wget -q -O- http://php-apache; done"
```

Step 5 — Watch replicas scale

Open a second terminal and observe the HPA respond.

```
kubectl get hpa php-apache --watch
```

✓ Confirm autoscaling

The **REPLICAS** column climbs as CPU rises above 50%, then settles back down a few minutes after you stop the load generator (Ctrl+C, then delete the pod).

Step 6 — Clean up the load generator

Stop the CPU load once you've confirmed scale-up.

```
kubectl delete pod load-generator
```

★ Key takeaway

HPA reacts to *actual measured usage*, not requests — that's why Metrics Server is a hard prerequisite. Scale-up is fast; scale-down is deliberately slower to avoid flapping.

End of Day 5 Lab Guide — nice work closing out the week.